

LLM-Assisted Repository-Level Generation with Structured Spec-Driven Engineering

Shuzhao Feng¹, Boqi Chen^{1,2}, Brett H Meyer¹, Gunter Mussbacher¹

8 July, 2026

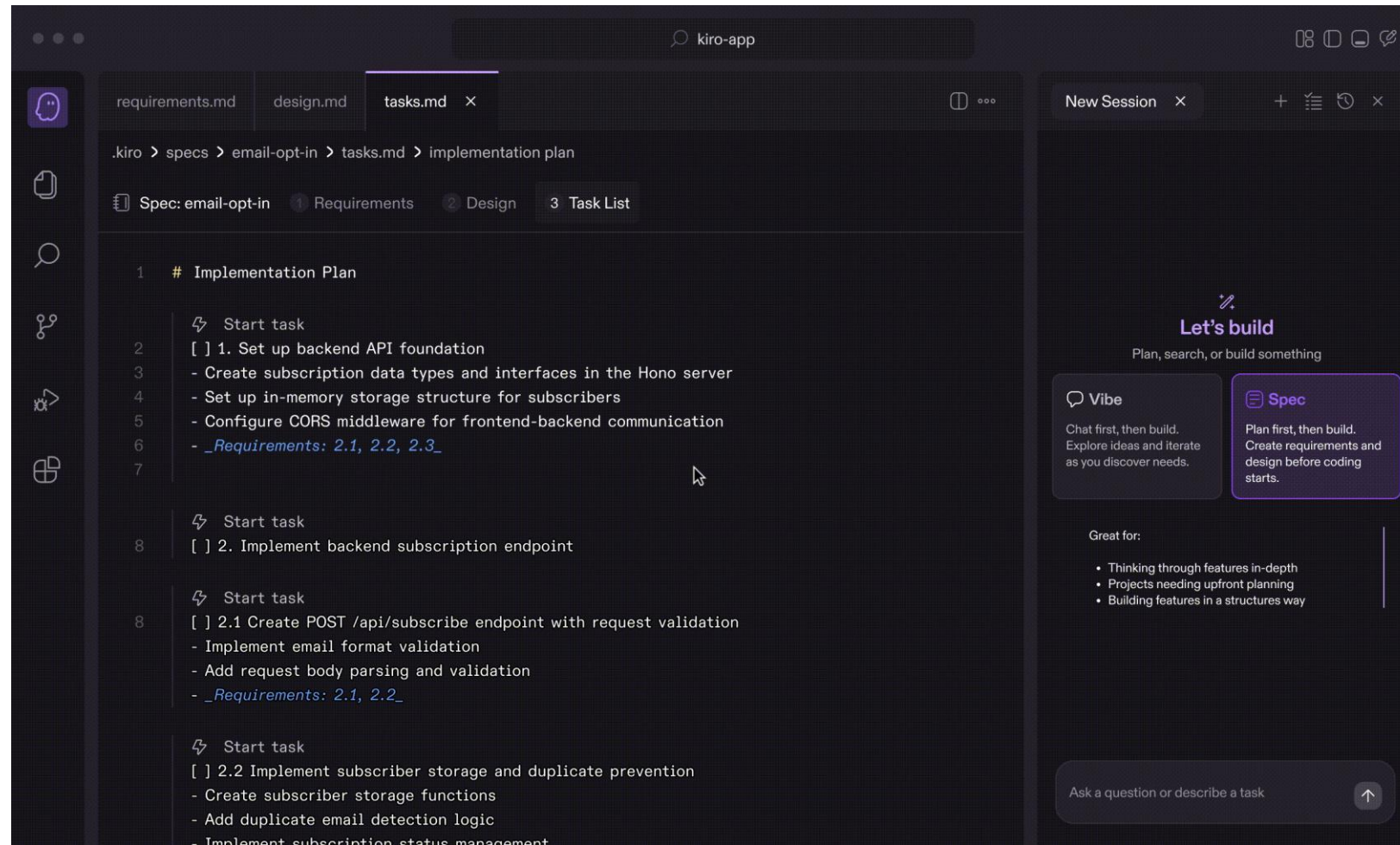
¹ Electrical and Computer Engineering, McGill University, Montreal, Canada

² Electrical Engineering and Computer Science, University of Ottawa, Ottawa, Canada

This research was supported by Natural Science and Engineering Research Council of Canada (NSERC)



LLM-Based Spec-Driven Development



Limitations

- Most Spec-Driven Engineering methods rely on **plain natural language** specifications
- Plain natural language is a “**lossy**” specification
- Plain natural language requirements aren’t systematically **verifiable**



If not plain natural language, then what?



Structured Specifications

- Specifications with a predefined **template**, e.g. class diagrams
- Design documents that often enables **automated verification**
- Commonly found in **traditional software engineering** practices

We Propose That....

By using structured specifications as LLM inputs instead of plain natural language, we can enable greater verifiability and significantly improve LLM's reliability in repository-level code generation.

Experiment Setup

- We designed experiments to build a **MVC (model-view-controller) system** using LLMs
- We generate the **business logic** (i.e. controllers) of the system
- We want to compare the performance of **structured specifications** against **plain natural language** as LLM input

Gherkin as Specifications

- Still natural language, but much more verifiable

```
search.feature x
1 Feature: Web Searching
2   In order to learn new stuff
3   As a Web surfer
4   I want to search for information
5
6   # Comments here
7
8   Scenario Outline: Search for animals
9     Given the Google home page is displayed
10    When the user searches for "<animal>"
11    Then search results are shown for "<animal>"
12
13    Examples: Animals
14      | animal |
15      | panda  |
16      | elephant |
17      | python |
18
```

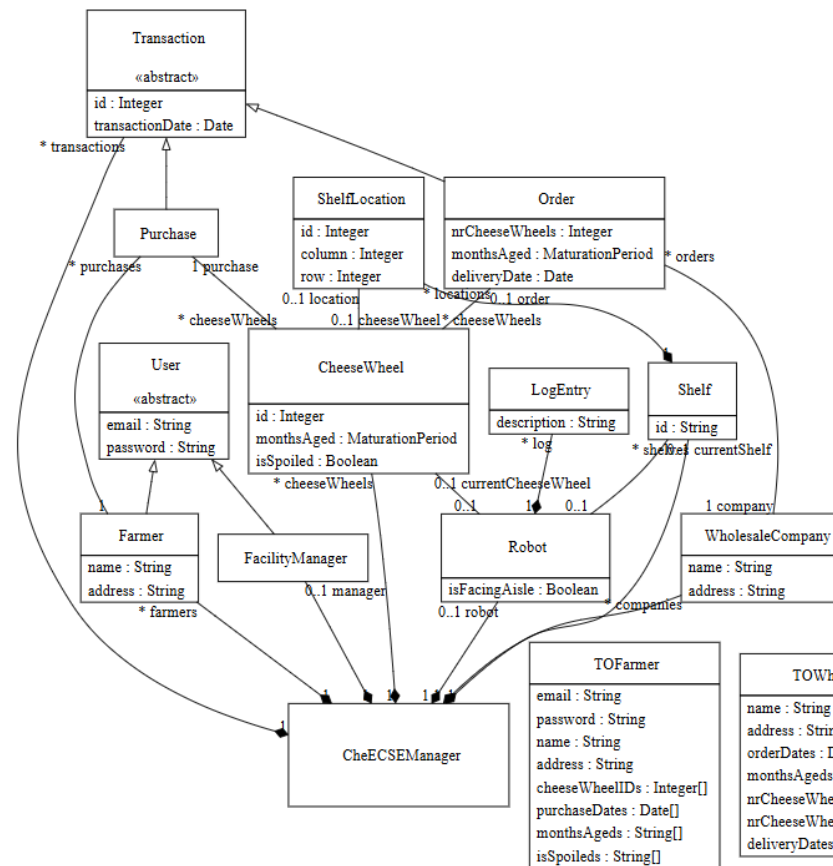
Domain Models as Backbone

• Domain modeling (example with Umlple)

```

1 namespace cheecsemanager;
2
3 class CheECSEManager {
4   1 <@>- 0..1 FacilityManager manager;
5   1 <@>- * Farmer farmers;
6   1 <@>- * Shelf shelves;
7   1 <@>- * CheeseWheel cheeseWheels;
8   1 <@>- * Transaction transactions;
9   1 <@>- * WholesaleCompany companies;
10  1 <@>- 0..1 Robot robot;
11 }
12
13 class User {
14   abstract;
15   unique immutable email;
16   password;
17 }
18
19 class FacilityManager {
20   isA User;
21 }
22
23 class Farmer {
24   depend cheecsemanager.CheeseWheel.MaturationPeriod;
25   isA User;
26   lazy name;
27   address;
28   1 -- * Purchase purchases;
29 }
30
31 class WholesaleCompany {
32   depend cheecsemanager.CheeseWheel.MaturationPeriod;
33   unique name;
34   address;
35   1 company -- * Order orders;
36 }
37
38 class Shelf {
39   unique id;
40   1 <@>- * ShelfLocation locations;
41 }

```



```

TOFarmer
email : String
password : String
name : String
address : String
cheeseWheelIDs : Integer[]
purchaseDates : Date[]
monthsAged : String[]
isSpoiled : Boolean

```

```

TOWholesaleCompany
name : String
address : String
orderDates : Date[]
monthsAged : String[]
nrCheeseWheelsOrdered : Integer[]
nrCheeseWheelsMissing : Integer[]
deliveryDates : Date[]

```

```

TOShelf
shelfID : String
cheeseWheelIDs : Integer[]
columnNrs : Integer[]
rowNrs : Integer[]
monthsAged : String[]

```

```

TOCheeseWheel
id : Integer
monthsAged : String
isSpoiled : Boolean
purchaseDate : Date
shelfID : String
column : Integer
row : Integer
isOrdered : Boolean

```

Domain Models as Backbone

• Domain modeling (example with Umpole)

```

1 namespace cheecsemanager;
2
3 class CheECSEManager {
4   1 <@>- 0..1 FacilityManager manager;
5   1 <@>- * Farmer farmers;
6   1 <@>- * Shelf shelves;
7   1 <@>- * CheeseWheel cheesewheels;
8   1 <@>- * Transaction transactions;
9   1 <@>- * WholesaleCompany companies;
10  1 <@>- 0..1 Robot robot;
11 }
12
13 class User {
14   abstract;
15   unique immutable email;
16   password;
17 }
18
19 class FacilityManager {
20   isA User;
21 }
22
23 class Farmer {
24   depend cheecsemanager.CheeseWheel.MaturationPeriod;
25   isA User;
26   lazy name;
27   address;
28   1 -- * Purchase purchases;
29 }
30
31 class WholesaleCompany {
32   depend cheecsemanager.CheeseWheel.MaturationPeriod;
33   unique name;
34   address;
35   1 company -- * Order orders;
36 }
37
38 class Shelf {
39   unique id;
40   1 <@>- * ShelfLocation locations;
41 }

```

```

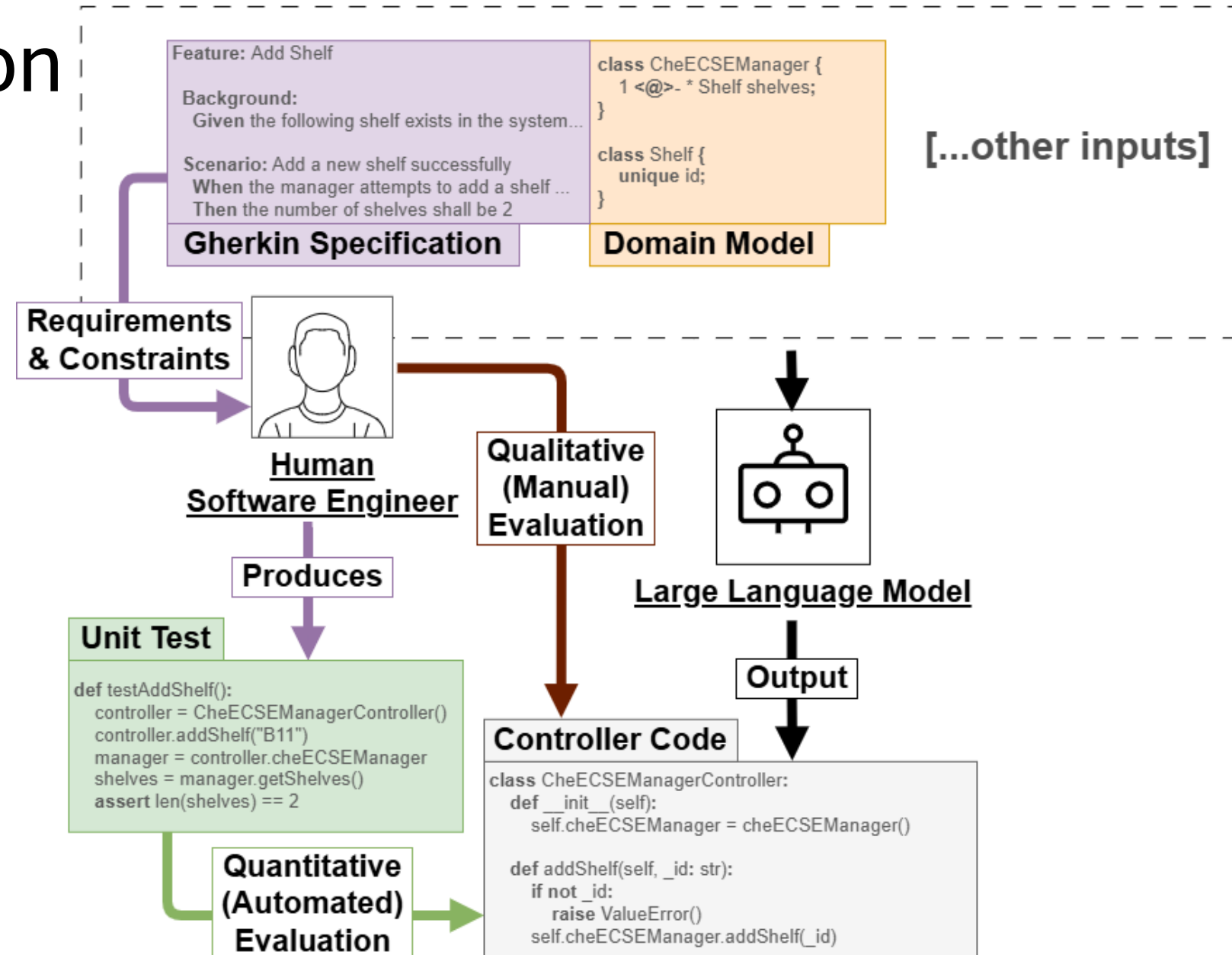
0001. # %% NEW FILE Order BEGINS HERE %%
0002.
0003. #PLEASE DO NOT EDIT THIS CODE
0004. #This code was generated using the UMPLE 1.37.0.8639.dcaf9c798 modeling language!
0005. # line 71 "../model.ump"
0006. from Transaction import Transaction
0007. import os
0008.
0009. class Order(Transaction):
0010.   #-----
0011.   # MEMBER VARIABLES
0012.   #-----
0013.   #Order Attributes
0014.   #Order Associations
0015.   #-----
0016.   # CONSTRUCTOR
0017.   #-----
0018.   def __init__(self, aTransactionDate, aCheECSEManager, aNrCheeseWheels, aMonthsAged, aDeliveryDate, aCompany):
0019.     self.cheeseWheels = None
0020.     self.company = None
0021.     self.deliveryDate = None
0022.     self.monthsAged = None
0023.     self.nrCheeseWheels = None
0024.     super().__init__(aTransactionDate, aCheECSEManager)
0025.     self.nrCheeseWheels = aNrCheeseWheels
0026.     self.monthsAged = aMonthsAged
0027.     self.deliveryDate = aDeliveryDate
0028.     didAddCompany = self.setCompany(aCompany)
0029.     if not didAddCompany:
0030.       raise RuntimeError("Unable to create order due to company. See https://manual.umpole.org/RE002ViolationofAssociationMultiplicity.html")
0031.     self.cheeseWheels = []
0032.
0033.   #-----
0034.   # INTERFACE
0035.   #-----
0036.   def setNrCheeseWheels(self, aNrCheeseWheels):
0037.     wasSet = False
0038.     self.nrCheeseWheels = aNrCheeseWheels
0039.     wasSet = True
0040.     return wasSet
0041.
0042.   def setMonthsAged(self, aMonthsAged):
0043.     wasSet = False
0044.     self.monthsAged = aMonthsAged
0045.     wasSet = True
0046.     return wasSet
0047.
0048.   def setDeliveryDate(self, aDeliveryDate):
0049.     wasSet = False
0050.     self.deliveryDate = aDeliveryDate
0051.     wasSet = True
0052.     return wasSet

```

Variants

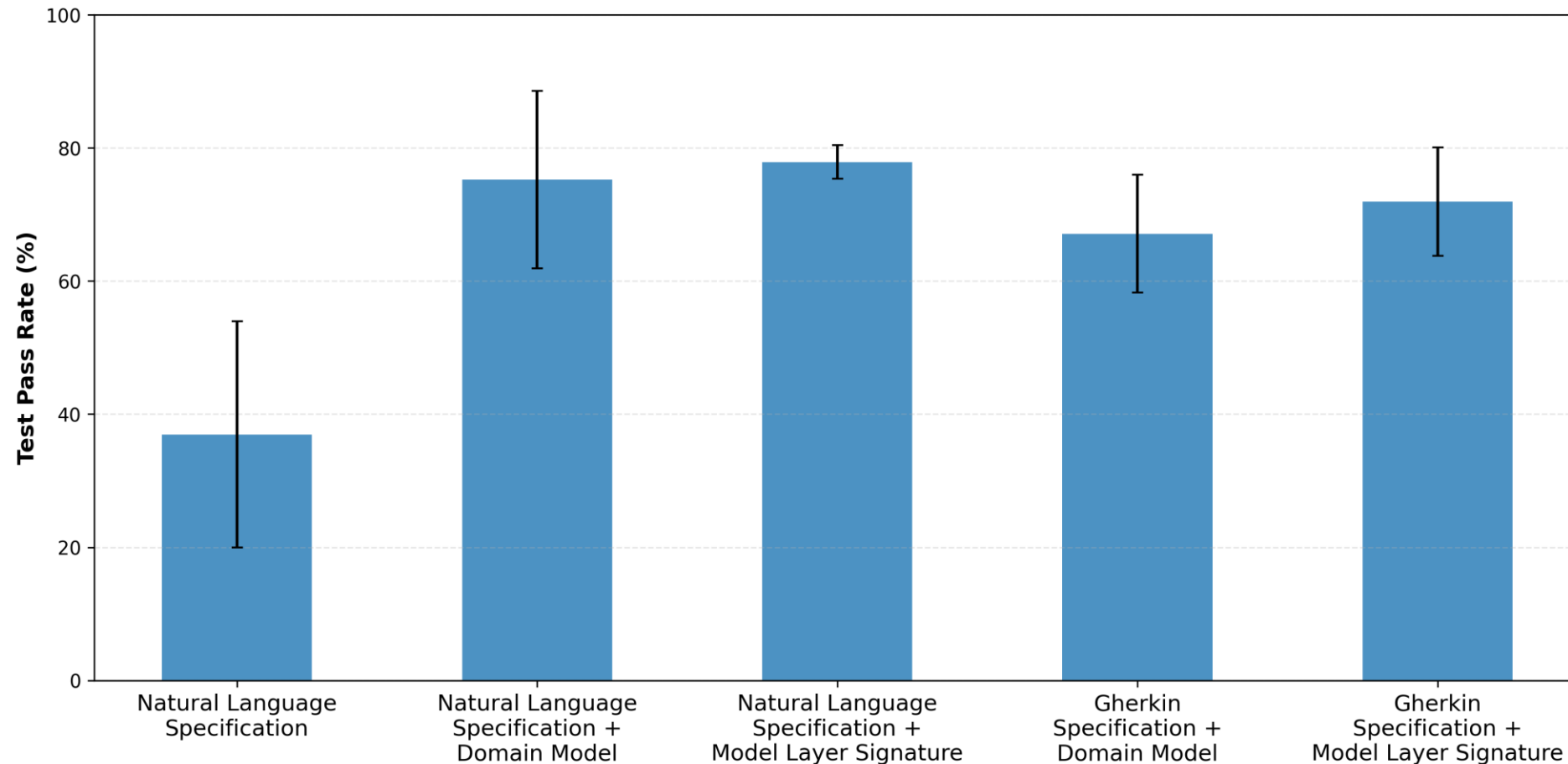
- Domain model variants
 - Umple: generated Python code is more **robust** with helper functions
 - Ecore: keep it **minimal**, only generate basic attributes
- Code-as-input variants
 - Domain model (only): **no Python code** as input, only domain modeling code
 - Model layer signature: The Python code **skeleton**, with function names and arguments, but no implementation

Validation

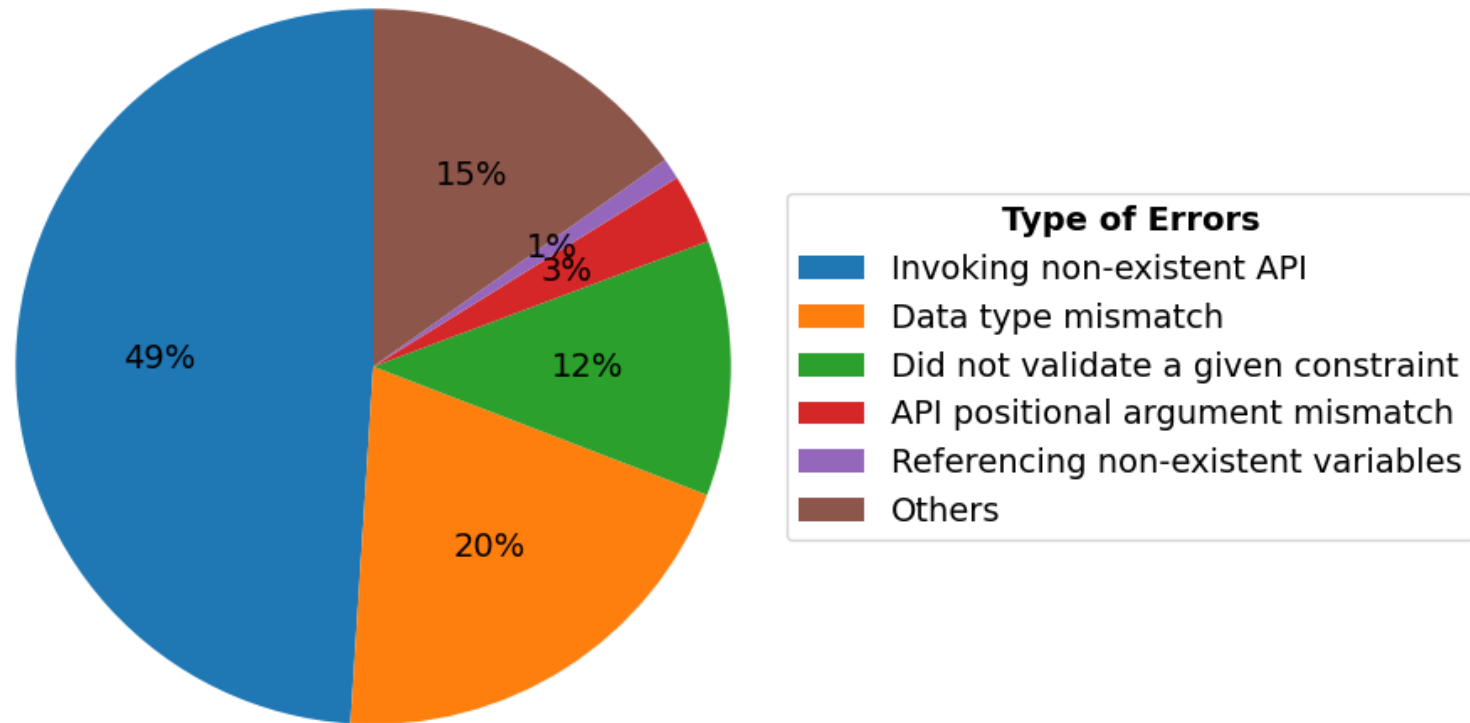


Results

Test Pass Rate across 10 Repetitions, using Claude Sonnet 4.5



Results



Next Steps

- Feedback loops, agentic harnesses
- Scale the experiments
- Better repository-level generation datasets
- Automated generation of structured specifications



<https://arxiv.org/abs/2605.02455>

**LLM-Assisted Repository-Level Generation
with Structured Spec-Driven Engineering**

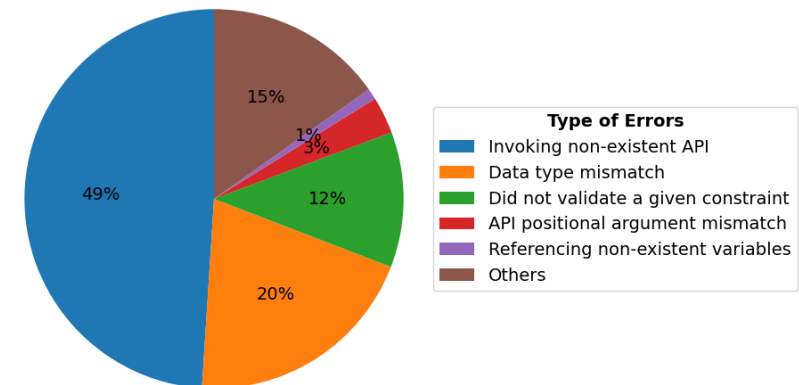
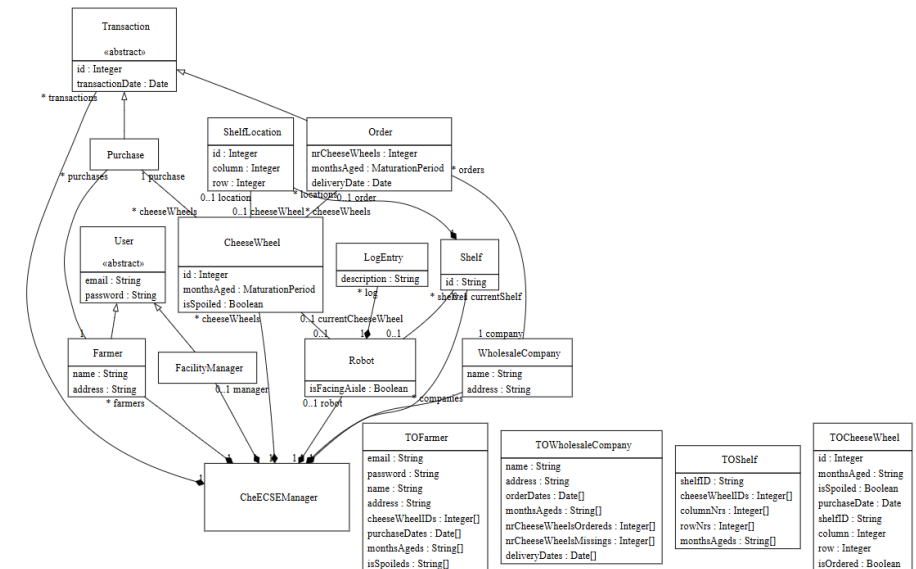
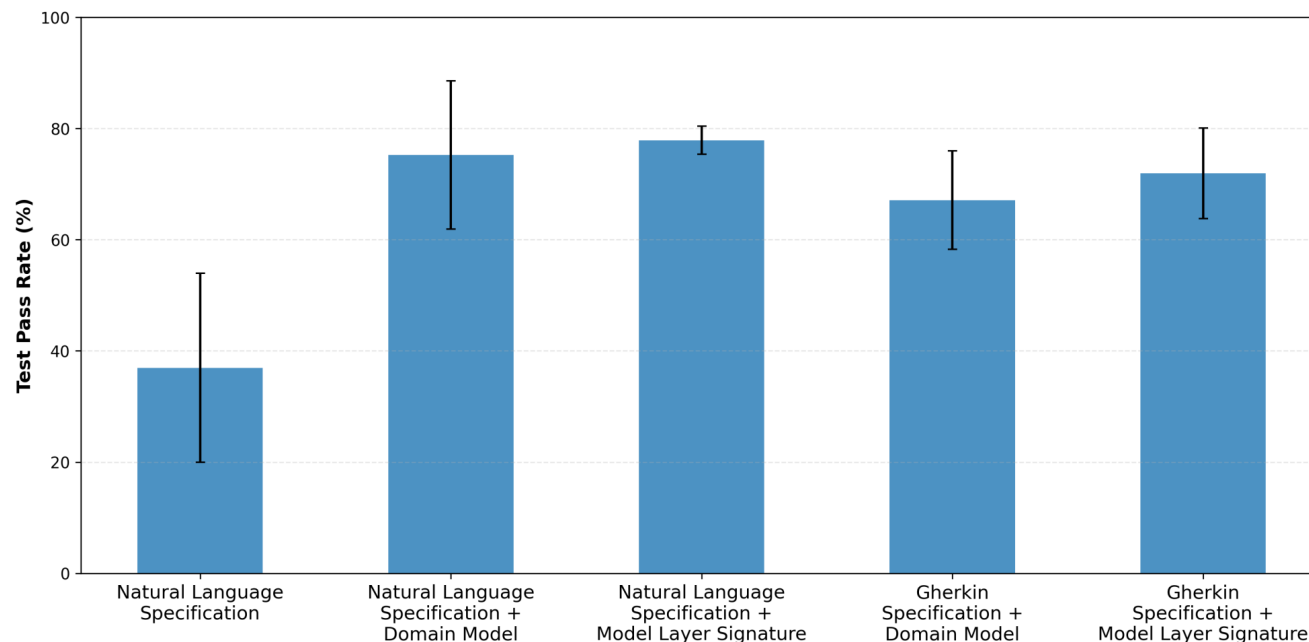


<https://shuzhaofeng.github.io/>

Shuzhao Feng

LLM-Assisted Repository-Level Generation with Structured Spec-Driven Engineering

- Most Spec-Driven Engineering methods rely on **plain natural language** specifications
- Plain natural language is a “**lossy**” specification
- Plain natural language requirements aren’t systematically **verifiable**



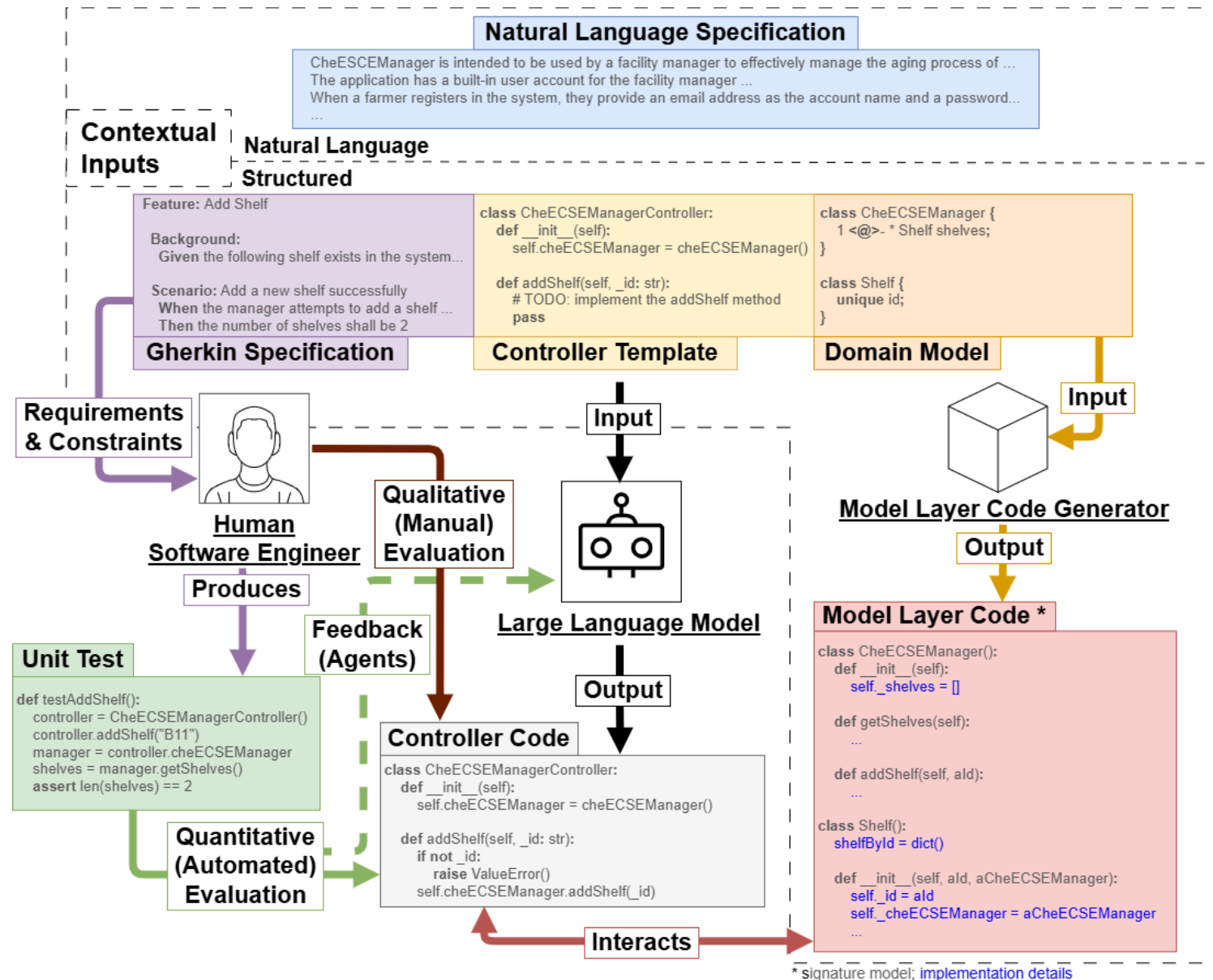


Table 1: MVC System Overview

	Symboleo	CheECSEManager	MeetingGroup
Num. Classes	12	17	29
Num. Enum Classes	6	1	12
Num. Attributes	12	48	88
Num. Relationships	26	20	55
Num. Test Cases	134	120	119

reference made by human

* details such as code comments are omitted

** the strike-through function does not exist

```
def create_driver(self, drivename: str):  
    if not drivename:  
        raise ValueError('The name of a driver cannot be empty.')
```

```
self.btms.addDriver(drivename)
```

gpt-4.1-nano, Controller signature + specification (i.e. no structural input) * **

```
def create_driver(self, drivename: str):  
    if not drivename:  
        raise ValueError('The name of a driver cannot be empty.')
```

```
self.btms.create_driver(drivename)
```

gpt-4.1-nano, Controller signature + full Python implementation (i.e. no natural language input) *

```
def create_driver(self, drivename: str):  
    self.btms.addDriver(drivename)
```

Constraint Handling

Model Layer Interaction

Table 2: Test Pass Rate of the Generated Business Logic with Claude Sonnet 4.5 using Various Input Configurations

System	Modeling Tool	Natural Language Specification			Gherkin Specification	
		(No Model)	Domain Model	Signature Model	Domain Model	Signature Model
Symboleo	Umple	0.0% $\pm$ 0.0%	79.9% $\pm$ 0.0%	79.9% $\pm$ 0.0%	<u>99.1% $\pm$ 2.9%</u>	79.9% $\pm$ 0.0%
	Ecore	47.9% $\pm$ 41.2%	79.9% $\pm$ 0.0%	79.9% $\pm$ 0.0%	<u>81.7% $\pm$ 6.1%</u>	79.9% $\pm$ 0.0%
CheECSEManager	Umple	43.5% $\pm$ 3.0%	73.0% $\pm$ 3.9%	76.7% $\pm$ 0.0%	25.7% $\pm$ 7.6%	<u>79.2% $\pm$ 0.3%</u>
	Ecore	16.8% $\pm$ 1.7%	53.2% $\pm$ 31.8%	<u>67.1% $\pm$ 4.0%</u>	26.0% $\pm$ 18.7%	28.2% $\pm$ 19.9%
MeetingGroups	Umple	81.6% $\pm$ 0.0%	82.2% $\pm$ 6.4%	<u>85.0% $\pm$ 2.8%</u>	83.4% $\pm$ 6.2%	84.6% $\pm$ 0.7%
	Ecore	31.9% $\pm$ 4.8%	82.4% $\pm$ 3.0%	78.8% $\pm$ 3.9%	<u>84.2% $\pm$ 0.9%</u>	79.8% $\pm$ 0.9%